Patrones de diseño en Python

Strategy · State · Iterator

Material práctico: un ejemplo resuelto y dos ejercicios propuestos por patrón.

1. Strategy — Ejemplo resuelto: calificaciones

Strategy permite cambiar el algoritmo utilizado por una clase sin modificarla.

```
from abc import ABC, abstractmethod

class EstrategiaEvaluacion(ABC):
    @abstractmethod
    def calcular(self, notas):
        pass

class PromedioSimple(EstrategiaEvaluacion):
    def calcular(self, notas):
        return sum(notas) / len(notas)

class PromedioPonderado(EstrategiaEvaluacion):
    def calcular(self, notas):
        return notas[0] * 0.20 + notas[1] * 0.30 + notas[2] * 0.50

class MejorNota(EstrategiaEvaluacion):
    def calcular(self, notas):
        return max(notas)

class Evaluador:
    def __init__(self, estrategia):
        self.estrategia = estrategia

    def cambiar_estrategia(self, estrategia):
        self.estrategia = estrategia

    def evaluar(self, notas):
        return self.estrategia.calcular(notas)

notas = [6, 8, 10]
evaluador = Evaluador(PromedioSimple())
print(evaluador.evaluar(notas))  # 8.0
evaluador.cambiar_estrategia(PromedioPonderado())
print(evaluador.evaluar(notas))  # 8.6
evaluador.cambiar_estrategia(MejorNota())
print(evaluador.evaluar(notas))  # 10
```

Ejercicio 1 — Cálculo de salarios

Crear EstrategiaSalario y las estrategias SalarioFijo, SalarioPorHoras y SalarioConComision. Crear CalculadoraSalario, capaz de cambiar de estrategia sin usar condicionales. Probar: fijo $500.000; 40 horas a $8.000 = $320.000; básico $300.000 más 10% de ventas por $1.000.000 = $400.000.

Ejercicio 2 — Sistema de pagos

Implementar PagoEfectivo (10% de descuento), PagoTransferencia (sin cambios) y PagoTarjeta (15% de recargo), todas con una interfaz común. Crear ProcesadorPago. Agregar luego PagoPromocional (20% de descuento) sin modificar ProcesadorPago.

2. State — Ejemplo resuelto: semáforo

State representa cada estado mediante un objeto. El comportamiento y las transiciones dependen del estado actual.

```
from abc import ABC, abstractmethod

class EstadoSemaforo(ABC):
    @abstractmethod
    def avanzar(self, semaforo):
        pass

class Verde(EstadoSemaforo):
    def avanzar(self, semaforo):
        print("Verde -> Amarillo")
        semaforo.cambiar_estado(Amarillo())

class Amarillo(EstadoSemaforo):
    def avanzar(self, semaforo):
        print("Amarillo -> Rojo")
        semaforo.cambiar_estado(Rojo())

class Rojo(EstadoSemaforo):
    def avanzar(self, semaforo):
        print("Rojo -> Verde")
        semaforo.cambiar_estado(Verde())

class Semaforo:
    def __init__(self):
        self.estado = Rojo()

    def cambiar_estado(self, nuevo_estado):
        self.estado = nuevo_estado

    def avanzar(self):
        print("Estado actual:", type(self.estado).__name__)
        self.estado.avanzar(self)

semaforo = Semaforo()
for i in range(5):
    semaforo.avanzar()
```

Nota: Rojo() crea un objeto aunque no tenga __init__ propio. En self.estado.avanzar(self), el primer self.estado es el objeto de estado; el argumento self es el Semaforo. El estado cambia mediante cambiar_estado(), sin necesidad de devolver otro objeto.

Ejercicio 3 — Reproductor de música

Implementar los estados Detenido, Reproduciendo y Pausado, con operaciones reproducir(), pausar() y detener(). Detenido + reproducir → Reproduciendo; Reproduciendo + pausar → Pausado; Pausado + reproducir → Reproduciendo; Reproduciendo o Pausado + detener → Detenido. Las acciones no permitidas deben informar el problema sin cambiar el estado.

Ejercicio 4 — Préstamo de biblioteca

Crear Libro con estados Disponible, Prestado y EnReparacion. Implementar prestar(), devolver(), enviar_reparacion() y finalizar_reparacion(). Un libro prestado debe devolverse antes de poder prestarlo otra vez; uno en reparación no puede prestarse. Cada estado decide qué acciones permite.

3. Iterator — Ejemplo resuelto: productos de una tienda

Iterator permite recorrer una colección sin acceder directamente a su estructura interna.

```
class Producto:
    def __init__(self, nombre, precio):
        self.nombre = nombre
        self.precio = precio

    def __str__(self):
        return f"{self.nombre}: ${self.precio}"

class IteradorProductos:
    def __init__(self, productos):
        self.productos = productos
        self.posicion = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.posicion >= len(self.productos):
            raise StopIteration
        producto = self.productos[self.posicion]
        self.posicion += 1
        return producto

class Tienda:
    def __init__(self):
        self.productos = []

    def agregar(self, producto):
        self.productos.append(producto)

    def __iter__(self):
        return IteradorProductos(self.productos)

tienda = Tienda()
tienda.agregar(Producto("Mouse", 15000))
tienda.agregar(Producto("Teclado", 25000))
tienda.agregar(Producto("Monitor", 180000))

for producto in tienda:
    print(producto)
```

Nota: __iter__() entrega el iterador; __next__() devuelve cada elemento. raise StopIteration señala al for que el recorrido terminó normalmente.

Ejercicio 5 — Alumnos aprobados

Crear Alumno (nombre, nota), Curso e IteradorAprobados. El iterador debe entregar únicamente alumnos con nota ≥ 6, sin filtrar en el programa principal. Probar: Ana 8, Luis 4, María 7, Pedro 3 y Sofía 9. Deben mostrarse Ana, María y Sofía.

Ejercicio 6 — Recorrido inverso de tareas

Crear Tarea, ListaTareas e IteradorInverso. Agregar las tareas “Estudiar Python”, “Preparar parcial” y “Entregar trabajo”. Recorrerlas desde la última hasta la primera mediante un for, sin acceder directamente a la lista interna desde el programa principal.